

Embedded C++

Nathan Warner



**Northern Illinois
University**

Computer Science
Northern Illinois University
United States

Contents

1	Threads and Thread safety	2
1.1	Threads and locks	2
1.2	Atomics	13
2	The Embedded Memory Model	27
3	Bit manipulations	31

Threads and Thread safety

1.1 Threads and locks

- Creating a thread

```
0  #include <iostream>
1  #include <thread>
2
3  void worker(int id) {
4      std::cout << "Worker " << id << '\n';
5  }
6
7  int main() {
8      std::thread t(worker, 1);
9
10     t.join();
11 }
```

The operating system schedules the new thread independently. You should not assume which thread runs first or how their instructions will be interleaved.

- **Join:** A `std::thread` is normally handled with `join()`.

```
0  std::thread t(do_work);
1  t.join();
```

`join()` blocks the calling thread until `t` finishes. A thread object must not be destroyed while it is still joinable. Doing so calls `std::terminate()`:

A thread must be joined to make the calling thread wait until the target thread finishes its work, prevent the main program from exiting too early, and safely clean up system resources

You can check its state:

```
0  if (t.joinable())
1      t.join();
```

- **Detached threads:**

```
0  t.detach();
```

A detached thread continues independently. The `std::thread` object no longer controls it.

Detached threads are dangerous because the program must ensure that everything the thread accesses remains alive:

```
0 void dangerous() {
1     int value = 42;
2
3     std::thread([&value] {
4         use(value); // value might already be destroyed
5     }).detach();
6 }
```

- **jthread**: C++20 introduced `std::jthread`. Its destructor automatically joins the thread.

```
0 #include <thread>
1
2 void worker() {
3     // Work
4 }
5
6 int main() {
7     std::jthread t(worker);
8 } // Automatically joined
```

- **Passing arguments**: Arguments passed to `std::thread` are normally copied or moved into internal storage.

```
0 void process(int value);
1
2 int x = 10;
3 std::thread t(process, x); // x is copied
```

To deliberately pass by reference, use `std::ref`:

```
0 #include <functional>
1
2 void increment(int& value) {
3     ++value;
4 }
5
6 int x = 10;
7 std::thread t(increment, std::ref(x));
8 t.join();
```

- **std::mutex**: A mutex provides mutual exclusion: only one thread can own it at a time.

```

0  #include <mutex>
1
2  int counter = 0;
3  std::mutex counter_mutex;
4
5  void increment()
6  {
7      for (int i = 0; i < 100'000; ++i) {
8          counter_mutex.lock();
9          ++counter;
10         counter_mutex.unlock();
11     }
12 }

```

The protected region is called a critical section. Do not normally call `lock()` and `unlock()` manually. If an exception or early return occurs before `unlock()`, the mutex remains locked

- **RAII locking:** C++ locking should normally use RAII: lock in a constructor, unlock in a destructor.

Use `std::lock_guard` for a simple scope-based lock:

```

0  void increment() {
1      for (int i = 0; i < 100'000; ++i) {
2          std::lock_guard<std::mutex> lock(counter_mutex);
3          ++counter;
4      }
5  }

```

The mutex is automatically unlocked at the end of the scope.

You can shorten the critical section:

```

0  void function(){
1      perform_unrelated_work();
2
3      {
4          std::lock_guard<std::mutex> lock(counter_mutex);
5          modify_shared_state();
6      }
7
8      perform_more_unrelated_work();
9  }

```

Keeping critical sections short reduces contention.

When you initialize a `std::lock_guard`, it calls `.lock()` on the underlying mutex (usually `std::mutex`). If another thread holds the lock, the operating system puts the requesting thread to sleep (context switch).

- **`std::unique_lock`**: `std::unique_lock` is more flexible:

```
0  std::unique_lock<std::mutex> lock(counter_mutex);
1
2  modify_shared_state();
3
4  lock.unlock();
5  perform_expensive_work();
6
7  lock.lock();
8  modify_shared_state_again();
```

It supports:

- Deferred locking
- Manual unlock/relock
- Ownership transfer
- Condition variables
- Timed mutex operations

It has slightly more state and overhead than `std::lock_guard`.

- **`std::scoped_lock`** is particularly useful when acquiring several mutexes:

```
0  std::scoped_lock lock(account_a.mutex, account_b.mutex);
```

It uses a deadlock-avoidance algorithm rather than naively locking them in the listed sequence.

- **`try_lock()` and timed mutexes**: `try_lock()` attempts to acquire a mutex without waiting:

```
0  if (mutex.try_lock()) {
1      // Mutex acquired
2      use_shared_state();
3      mutex.unlock();
4  } else {
5      // Do something else
6  }
```

The primary difference is that `std::mutex::lock()` is a blocking operation, while `std::mutex::try_lock()` is a non-blocking operation

RAII can still be used:

```

0  std::unique_lock lock(mutex, std::try_to_lock);
1
2  if (lock.owns_lock()) {
3      use_shared_state();
4  }

```

- **Condition variables:** In C++, `std::condition_variable` is a synchronization primitive that allows one thread to suspend execution (sleep) until another thread modifies a shared variable and signals the condition variable. This eliminates the need for inefficient busy-waiting loops

To use a condition variable, you need three pieces working together

1. **`std::condition_variable`:** The signaling mechanism.
2. **`std::mutex`:** Protects the shared state.
3. **`std::unique_lock<std::mutex>`:** Passed to the `wait()` function. A `std::lock_guard` cannot be used for waiting because the condition variable must frequently unlock and relock the mutex while waiting

```

0  #include <condition_variable>
1  #include <mutex>
2  #include <queue>
3
4  std::queue<int> queue;
5  std::mutex mutex;
6  std::condition_variable available;
7  bool finished = false;

```

```

0  // Producer
1  void produce(int value) {
2      {
3          std::lock_guard lock(mutex);
4          queue.push(value);
5      }
6
7      available.notify_one();
8  }

```

```

0  // Consumer
1
2  void consume() {
3      for (;;) {
4          std::unique_lock lock(mutex);
5
6          available.wait(lock, [] {
7              return !queue.empty() && finished;
8          });
9
10         if (queue.empty() && finished) {
11             break;
12         }
13
14         int value = queue.front();
15         queue.pop();
16
17         lock.unlock();
18         process(value);
19     }
20 }

```

The predicate must be checked because:

- Wakeups can be spurious.
- Another consumer may take the item first.
- A notification means the state may have changed, not that the condition is definitely true.

Note: A spurious wakeup occurs when a thread wakes up from a waiting state (sleeping/blocked on a condition variable or monitor) without receiving any explicit signal, notification, or interrupt, and before the condition it is waiting for has actually been met.

The actual condition is the protected shared state, such as `!queue.empty()`. The condition variable is only the notification mechanism.

- **std::shared_lock:** `std::shared_lock` is a general-purpose, RAII-compliant shared lock ownership wrapper introduced in C++14. It is primarily designed for reader-writer scenarios, allowing multiple "reader" threads to concurrently access a shared resource while ensuring that "writer" threads get exclusive access
- **Reader-writer locking:** `std::shared_mutex` permits:
 - Multiple simultaneous readers
 - One exclusive writer

`std::shared_mutex` permits:

- Multiple simultaneous readers
- One exclusive writer
- **Multiple Readers:** Multiple threads can hold a `std::shared_lock` on the same mutex simultaneously

- **Exclusive Block:** If any thread holds an exclusive lock (e.g., via `std::unique_lock` or `std::lock_guard`), all `std::shared_lock` requests will block until the exclusive lock is released
- **RAII Management:** It automatically calls `lock_shared()` on construction and `unlock_shared()` upon destruction (when it goes out of scope)

```

0  #include <shared_mutex>
1  #include <unordered_map>
2
3  class Dictionary {
4      public:
5          int get(int key) const {
6              std::shared_lock lock(mutex_);
7              return values_.at(key);
8          }
9
10         void set(int key, int value) {
11             std::unique_lock lock(mutex_);
12             values_[key] = value;
13         }
14
15         private:
16             mutable std::shared_mutex mutex_;
17             std::unordered_map<int, int> values_;
18     };

```

This helps primarily when reads are frequent, writes are uncommon, and the protected operations are substantial enough to justify the added complexity. An ordinary mutex may otherwise perform better.

- **One-time initialization:** Use `std::call_once` when initialization must occur exactly once:

```

0  #include <mutex>
1
2  std::once_flag initialization_flag;
3
4  void initialize()
5  {
6      std::call_once(initialization_flag, [] {
7          initialize_library();
8      });
9  }

```

Function-local static initialization is also thread-safe since C++11:

```

0  Resource& resource() {
1      static Resource instance;
2      return instance;
3  }

```

- **Semaphore:** A semaphore represents a number of available permits:

```
0  #include <semaphore>
1
2  std::counting_semaphore<4> slots(4);
3
4  void worker() {
5      slots.acquire();
6      use_limited_resource();
7      slots.release();
8  }
```

Unlike a mutex, a semaphore does not represent ownership of a critical section. It is useful for limiting concurrency or representing available resources.

A semaphore (whether a counting semaphore or a binary semaphore) does not represent ownership because it is a signaling mechanism, not a locking mechanism.

Unlike a mutex (mutual exclusion lock), which is explicitly designed to manage ownership of a critical section, a semaphore focuses strictly on managing a count of available resources.

- **The Rule:** In a mutex, only the exact thread that locked it can unlock it.
- **The Semaphore Difference:** Any thread can call `signal()` (or `post/V`) on a semaphore, even if it never called `wait()` (or `pend/P`) to acquire it. Because there is no restriction tying a specific thread to the resource, no concept of "ownership" exists.

- **Latch:** A `std::latch` is a one-use countdown:

```
0  #include <latch>
1  #include <thread>
2  #include <vector>
3
4  std::latch completed(4);
5
6  void worker() {
7      perform_work();
8      completed.count_down();
9  }
10
11 // Start four workers...
12 completed.wait();
```

`std::latch` is a C++20 thread synchronization primitive that pauses one or more threads until a downward counter reaches zero

- **Single-Use:** Once the internal counter hits zero, the latch is done and cannot be reset or reused.
- **No Increment:** You cannot increase the counter after creation; you only count it down.

`std::latch::wait` blocks the calling thread until the internal counter of the `std::latch` reaches zero.

```

0  #include <latches>
1  #include <thread>
2  #include <vector>
3
4  // Initialize latch with count for 3 workers
5  std::latch work_latch(3);
6
7  for (int i = 0; i < 3; ++i) {
8      std::thread([&work_latch]() {
9          // Do work...
10         work_latch.count_down(); // Signal completion
11     }).detach();
12 }
13
14 work_latch.wait(); // Main thread blocks here until all 3
    ↪ workers finish

```

- **std::barrier:** std::barrier is a C++20 thread-coordination mechanism that blocks a group of participating threads until a specific number of arrivals are reached, after which it can be reused for multiple phases.

Unlike std::latch, which is single-use and destroys its counter value at zero, std::barrier automatically resets and can be used for repetitive coordination cycles

When all threads arrive at the synchronization point, the barrier executes a completion step (callable object) on one of the participating threads before unblocking the rest

```

0  #include <barrier>
1  int main() {
2      // 1. Define a completion function to run at the end of
   ↳ each phase
3      auto on_completion = []() noexcept {
4          ...
5      };
6
7      std::barrier sync_point(2, on_completion);
8
9      auto worker = [&](std::string name) {
10         ...
11
12         // Block until all 2 threads reach this point
13         sync_point.arrive_and_wait();
14
15         ...
16
17         // Reuse the same barrier for the next phase
18         sync_point.arrive_and_wait();
19
20         ...
21     };
22
23     std::vector<std::thread> threads;
24     threads.emplace_back(worker, "Thread A");
25     threads.emplace_back(worker, "Thread B");
26
27     for (auto& t : threads) {
28         t.join();
29     }
30
31     return 0;
32 }

```

- **std::async:** std::async is a function template in C++ (introduced in C++11) that runs a callable object—like a function, lambda, or functor—asynchronously or lazily, returning a std::future that will eventually hold the result. It abstracts away manual thread management, automatically handling execution and allowing you to retrieve return values or handle exceptions across thread boundaries safely
- **std::promise:** A std::promise represents a commitment made by a worker thread to provide a result at some point. The thread fulfills this contract by calling .set_value().
- **std::future:** A std::future represents a placeholder for a value that does not exist yet, but will exist in the future.

When you call .get(), you are looking into the "future." If the future hasn't arrived yet, you sit and wait for it.

- **std::promise with std::future:** In C++, std::promise and std::future are a pair of class templates introduced in C++11 to pass values, exceptions, or signals between threads without explicitly managing mutexes or condition variables. They establish a one-way, single-use communication channel where one thread acts as the producer and another acts as the consumer

```

0  void calculate(std::promise<int> result)
1  {
2      try {
3          result.set_value(42);
4      } catch (...) {
5          result.set_exception(std::current_exception());
6      }
7  }
8
9  int main()
10 {
11     std::promise<int> promise;
12     std::future<int> future = promise.get_future();
13
14     std::thread t(calculate, std::move(promise));
15
16     int result = future.get();
17     t.join();
18 }

```

Promises are useful when the producer and consumer need a direct one-time communication channel.

1.2 Atomics

- **std::atomic<T>**: std::atomic<T> provides thread-safe access to a shared value without requiring an explicit mutex for each operation.

Its main purpose is preventing a data race when multiple threads read or modify the same variable.

Without an atomic variable, this program has undefined behavior

```
0  #include <thread>
1
2  int counter = 0;
3
4  void increment()
5  {
6      for (int i = 0; i < 100'000; ++i) {
7          ++counter;
8      }
9  }
```

++counter is a read-modify-write operation. Two threads can interfere with each other. Instead, we can use std::atomic

```
0  #include <atomic>
1  #include <iostream>
2  #include <thread>
3
4  std::atomic<int> counter{0};
5
6  void increment()
7  {
8      for (int i = 0; i < 100'000; ++i) {
9          ++counter;
10     }
11 }
12
13 int main()
14 {
15     std::thread first(increment);
16     std::thread second(increment);
17
18     first.join();
19     second.join();
20
21     std::cout << counter.load() << '\n'; // 200000
22 }
```

Every increment is now indivisible from the perspective of other threads.

- **Operations:**

```

0  std::atomic<int> value{10};
1
2  value.store(20);           // Atomic write
3  int x = value.load();     // Atomic read
4
5  value.fetch_add(5);        // Add 5; returns the old value
6  value.fetch_sub(2);        // Subtract 2; returns the old
   ↪ value
7
8  value++;                   // Atomic increment
9  value--;                   // Atomic decrement

```

- **Cannot be copied:** Atomic objects generally cannot be copied. You copy the contained value explicitly
- **Atomic flags:** A common use is communicating a simple condition between threads:

```

0  #include <atomic>
1  #include <thread>
2
3  std::atomic<bool> stop_requested{false};
4
5  void worker()
6  {
7      while (!stop_requested.load()) {
8          // Perform work
9      }
10 }
11
12 int main()
13 {
14     std::thread thread(worker);
15
16     // Later:
17     stop_requested.store(true);
18
19     thread.join();
20 }

```

Atomicity ensures that accessing `stop_requested` does not create a data race. If `stop_requested` were a normal `bool`, the worker thread would read it while the main thread writes it. One thread writes the object while another thread reads it, producing a data race. In C++, a data race results in undefined behavior.

Because ordinary C++ code has no defined data races, the compiler may assume another thread does not modify `stop_requested`. It could effectively transform this:

```

0  while (!stop_requested) {
1  }

```

into something resembling:

```
0  bool cached = stop_requested;
1
2  while (!cached) {
3  }
```

The worker might therefore loop forever even after the main thread assigns true.

Each CPU core has registers, caches, and store buffers. A normal write does not provide the synchronization required by the C++ memory model to make another thread observe the change correctly.

Modern hardware has cache-coherence mechanisms, but those mechanisms alone do not make racy C++ code valid. The compiler and processor are both allowed to reorder ordinary operations when no synchronization is present.

On many systems, reading or writing a single byte-sized bool happens as one hardware operation. That only means the reader probably will not observe a partially written value.

Thread safety requires more than preventing a torn read. It also requires:

- Defined concurrent access
- Appropriate visibility between threads
- Restrictions on compiler and processor reordering

A normal bool provides none of these guarantees.

- **Torn read:** A torn read is a concurrency bug that occurs when a thread reads a value from memory while another thread is simultaneously writing to it, resulting in the reader seeing a partially updated, corrupted value
- **Cache coherence:** Cache coherence is the practice of making sure that all copies of shared data stored in different local caches remain identical in a multi-core or multi-processor system
- **Compare-and-exchange:** Compare-and-exchange is central to lock-free algorithms. It means if the atomic currently contains the expected value, replace it with the desired value. Otherwise, update the expected variable with the actual value.

```
0  std::atomic<int> value{10};
1
2  int expected = 10;
3
4  bool changed = value.compare_exchange_strong(expected, 20);
```

- **strong vs weak compare exchange:** `compare_exchange_weak` and `compare_exchange_strong` perform a conditional atomic update called compare-and-exchange, often abbreviated CAS.

Conceptually, CAS performs this indivisible operation:

```
0  if (atomic_value == expected) {
1      atomic_value = desired;
2      return true;
3  } else {
4      expected = atomic_value;
5      return false;
6  }
```

No other thread can interrupt the comparison and replacement as two separate steps.

Consider

```
0  int current = value.load();
1  value.store(current + 1);
```

Two threads could perform this sequence:

Step	Thread A	Thread B	Atomic value
1	Loads 10	10	
2	Loads 10	10	
3	Stores 11	11	
4	Stores 11	11	

One increment is lost. CAS allows each thread to store the new value only if the old value has not changed since it was loaded

The fundamental difference is that `compare_exchange_weak` may report failure even when the atomic object equals expected. This is called a **spurious failure**

A **spurious failure** in a compare-and-exchange (or Compare-and-Swap / CAS) operation occurs when the function returns false (indicating failure) even though the current value in memory exactly matches the expected value. Instead of executing the swap, the operation acts as if the values did not match

On many RISC architectures (like ARM, PowerPC, or MIPS), atomic operations are implemented using a pair of instructions: Load-Link (LL) and Store-Conditional (SC)

The LL instruction tracks the memory address. The SC instruction only succeeds if no changes or specific hardware events occurred at that address since the LL instruction.

Context switches, system interrupts, or even basic CPU cache eviction events can cause the CPU to "lose the reservation" on that memory address. As a result, the SC instruction fails naturally, inducing a spurious failure even if no other thread changed the underlying value

Strong fails only when the comparison fails, it is suitable for a one-time attempt.

Weak may fail because

- The atomic value did not equal expected; or
- The atomic value did equal expected, but the implementation produced a spurious failure.

Using `compare_exchange_strong` in a retry loop is usually considered a performance anti-pattern because `compare_exchange_weak` is more efficient on many CPU architectures.

While `compare_exchange_strong` guarantees it will only fail if the value actually changed, this guarantee comes with a hardware cost that is unnecessary inside a loop.

So, weak is normally used in a loop.

- **Memory ordering:** Atomic operations have two related responsibilities:

1. Making an individual operation indivisible.
2. Controlling how memory operations become visible between threads.

Memory ordering defines how compilers and CPU hardware are allowed to reorder read and write operations across multiple threads

- **Single-threaded code:** Compilers and CPUs change instruction order to make code run faster. This reordering is invisible because the final result stays the same.
 - **Multi-threaded code:** Other threads can spot these reordered operations. Without rules, one thread might see updates in a completely different order than another thread wrote them, breaking synchronization
- **Store-Buffer Forwarding (Store-Load Reordering):** Imagine two threads running on two different CPU cores, sharing two variables (X and Y), both starting at 0.

Thread 1 (Core 1)	Thread 2 (Core 2)
1. Write 1 to X	3. Write 1 to Y
2. Read Y	4. Read X

Logically, it should be impossible for both threads to read 0. At least one thread has to write its 1 before the other thread reads.

Yet, without memory barriers, both reads can return 0.

Writing data to the main RAM (or even the L1 cache) is slow. To speed things up, each CPU core has a tiny, ultra-fast temporary holding zone called a Store Buffer.

1. Core 1 executes Write 1 to X. Instead of waiting for X to travel to cache, Core 1 drops the write into its local Store Buffer and immediately moves on.
2. Core 1 executes Read Y. It checks its own Store Buffer (nothing for Y there) and reads Y from the cache. Because Core 2 hasn't flushed its own buffer yet, Y is still 0.

3. Core 2 does the exact same thing simultaneously. It drops Write 1 to Y into its own Store Buffer and immediately executes Read X, which returns 0.
4. Finally, both cores flush their Store Buffers out to the shared cache.

To stop the CPU from doing this, you must use a Memory Barrier (like C++'s `std::memory_order_seq_cst`). This instruction forces the CPU core to completely empty its local Store Buffer and commit the data to shared cache before it is allowed to perform the subsequent Read.

- **Memory barrier:** A memory barrier (also called a memory fence) is a specific CPU instruction that forces the processor and the compiler to maintain a strict order of memory operations. It acts as a wall that prevents reads and writes from crossing past it.

Barrier Type	What It Does	Common Use Case
LoadLoad	Ensures older reads finish before newer reads can start.	Preventing a thread from reading stale data.
StoreStore	Ensures older writes are visible to cache before newer writes happen.	Guaranteeing a payload is written before turning on a "Ready" flag.
LoadStore	Ensures older reads finish before newer writes can happen.	Preventing early overwrites of data.
StoreLoad	Ensures older writes are completely flushed before newer reads start.	The heaviest barrier. Required to prevent the store-buffer shortcut described earlier.

- **More on memory ordering:** Memory ordering controls how operations surrounding an atomic operation may be observed by other threads.

Every `std::atomic` operation is atomic regardless of its memory order. The ordering argument determines whether that operation also establishes ordering and visibility for other memory accesses.

Consider

```

0  int data = 0;
1  std::atomic<bool> ready{false};
2
3  void producer() {
4      data = 42;
5      ready.store(true);
6  }
7
8  void consumer() {
9      while (!ready.load()) { }
10
11     std::cout << data << '\n';
12 }

```

The intent is:

- Producer writes data.
- Producer sets ready.
- Consumer observes ready.
- Consumer reads data.

For this to work, the atomic flag must do more than safely transfer a Boolean value. It must also ensure that the write to data becomes visible before the consumer reads it. Memory ordering defines that relationship.

- **Program order is not necessarily observation order:** Within one thread, the source code says:

```
0  data = 42;  
1  ready = true;
```

But several layers may reorder or delay operations:

- The compiler may rearrange instructions.
- The CPU may execute or retire memory operations out of order.
- Stores may temporarily remain in a store buffer.
- Different cores may observe stores at different times.

These transformations are allowed when they do not change the observable behavior of a correctly synchronized C++ program.

Atomic memory orders constrain these transformations.

Importantly, the C++ memory model is the authority. Hardware cache coherence alone does not make unsynchronized C++ code valid.

- **Sequenced-before:** ordering inside one thread.
 - **Synchronizes-with:** ordering established between threads through synchronization operations.
 - **Happens-before:** the combined ordering that makes earlier effects visible and prevents data races.
- **Global order:** A global order in thread memory models is a single, total sequence of operations that every thread in a program agrees on and observes in the exact same way

If Thread A performs an operation before Thread B in this global sequence, every other thread in the system will witness the events in that exact same chronological order

CPUs and compilers cannot reorder operations across different threads in a way that violates this shared global sequence

It mimics a simple single-threaded program where instructions happen one after another, making concurrent code easy to reason about

- **Even more on memory ordering:** `std::memory_order` controls two separate properties of an atomic operation:
 1. The atomic operation itself: whether other threads can observe it partially.
 2. The ordering of other memory operations around it: whether ordinary reads and writes become visible across threads.

Every `std::atomic` operation is atomic regardless of its memory order. The difference is synchronization.

Order	Atomicity	Acquires earlier writes	Publishes earlier writes	Global order
‘relaxed’	Yes	No	No	No
‘consume’	Yes	Dependency-based historically	No	No
‘acquire’	Yes	Yes	No	No
‘release’	Yes	No	Yes	No
‘acq_rel’	Yes	Yes	Yes	No
‘seq_cst’	Yes	Yes	Yes	Yes, among sequentially consistent operations

Note that modern spelling is

```
0  std::memory_order::relaxed
1  std::memory_order::acquire
2  std::memory_order::release
3  std::memory_order::acq_rel
4  std::memory_order::seq_cst
```

The older spellings, such as `std::memory_order_relaxed`, are also commonly seen.

- **Release and acquire:**

```
0  int data = 0;
1  std::atomic<bool> ready{false};
```

Thread 1:

```
0  data = 42;
1  ready.store(true, std::memory_order::release);
```

Thread 2:

```
0 while (!ready.load(std::memory_order::acquire)) {  
1 }  
2  
3 std::cout << data; // Guaranteed to print 42
```

1. data = 42 sequenced-before
2. release store of true synchronizes-with
3. acquire load that reads true sequenced-before
4. reading data

Therefore, writing data happens-before reading data.

The critical condition is that the acquire load must observe the value from the release store. Merely using release and acquire somewhere in the program is insufficient.

if the load reads false, the consumer keeps waiting. When it eventually reads the producer's true, the release/acquire synchronization is established.

“Sequenced-before” describes ordering within one thread. Because statement A appears before statement B in the producer thread, A is sequenced-before B:

The release store effectively publishes everything that the producer did before it. Therefore, the write to data is part of what is being published.

- **memory_order_relaxed**: Relaxed ordering guarantees atomicity and a per-object modification order, but it creates no synchronization for other memory.

This is appropriate when you only need an accurate counter. Concurrent increments cannot overwrite each other.

However, relaxed ordering cannot normally publish surrounding data:

```
0 int data = 0;  
1 std::atomic<bool> ready{false};  
2  
3 // Thread 1  
4 data = 42;  
5 ready.store(true, std::memory_order::relaxed);  
6  
7 // Thread 2  
8 if (ready.load(std::memory_order::relaxed)) {  
9     std::cout << data; // Data race: undefined behavior  
10 }
```

The atomic ready variable is safe, but nothing connects the ordinary write to data with the ordinary read from data.

- **memory_order_release**: Release ordering is used on stores and read-modify-write operations.

```
0 ready.store(true, std::memory_order::release);
```

Its purpose is publication: operations sequenced before the release can become visible to a thread that performs a matching acquire operation.

Valid operations are

```
0 atomic.store(value, std::memory_order::release);
1 atomic.exchange(value, std::memory_order::release);
2 atomic.fetch_add(value, std::memory_order::release);
```

This is invalid:

```
0 atomic.load(std::memory_order::release); // Invalid
```

A load does not publish a value, so release ordering does not make sense for a pure load.

- **memory_order_acquire**: Acquire ordering is used on loads and read-modify-write operations.

```
0 bool value = ready.load(std::memory_order::acquire);
```

If this operation reads from a corresponding release operation, everything published before that release becomes visible after the acquire.

An acquire load does not publish preceding operations to other threads. It receives information published by another thread.

A pure store does not read a value, so it cannot acquire anything.

- **memory_order_acq_rel**: Acquire-release combines both directions:
 - Acquire the writes published by another thread.
 - Release the current thread's preceding writes.

It is intended for atomic read-modify-write operations:

```
0 atomic.fetch_add(1, std::memory_order::acq_rel);
1 atomic.exchange(value, std::memory_order::acq_rel);
2 atomic.compare_exchange_weak(
3     expected,
4     desired,
5     std::memory_order::acq_rel
6 );
```

A read-modify-write operation both reads the old value and publishes a new value, so it can meaningfully perform both functions.

```
0  std::atomic<int> state{0};
1
2  int old = state.fetch_add(1, std::memory_order::acq_rel);
```

The operation can:

- Acquire information associated with the value it reads.
- Release preceding work through the new value it writes.

This is invalid for pure loads and stores

```
0  atomic.load(std::memory_order::acq_rel);    // Invalid
1  atomic.store(1, std::memory_order::acq_rel); // Invalid
```

Use acquire for the load and release for the store.

- **memory_order_seq_cst**: Sequential consistency provides acquire/release behavior plus a single total order over all sequentially consistent atomic operations.

```
0  atomic.store(1); // seq_cst
1  int x = atomic.load(); // seq_cst
```

The program behaves as though all `seq_cst` operations were placed into one global sequence that every thread agrees upon. The standard requires a single total order for these operations

Consider

```
0  std::atomic<int> x{0};
1  std::atomic<int> y{0};
2
3  int r1;
4  int r2;
```

```
0  // Thread 1
1  x.store(1, std::memory_order::seq_cst);
2  r1 = y.load(std::memory_order::seq_cst);
3
4  // Thread 2
5  y.store(1, std::memory_order::seq_cst);
6  r2 = x.load(std::memory_order::seq_cst);
```

With exclusively sequentially consistent operations, this result is forbidden:

```
0  r1 == 0 && r2 == 0
```

To obtain both zeros, each load would need to appear before the other thread's store, producing an impossible cycle in the global sequentially consistent order.

- **Release/acquire synchronization is conditional:** An acquire load synchronizes with a release store only when the load reads the value written by that store.

So, with

```
0  x.store(1, std::memory_order::release);
1  r1 = y.load(std::memory_order::acquire);
2
3  // Thread 2
4  y.store(1, std::memory_order::release);
5  r2 = x.load(std::memory_order::acquire);
6
7  // r1 == r2 == 0 possible
```

For thread 1's load

```
0  r1 = y.load(std::memory_order::acquire);
```

- If it reads 1, it read from Thread 2's release store, so synchronization occurs.
- If it reads the original 0, it did not read from Thread 2's release store, so no synchronization occurs.

The same applies to Thread 2's load of x. Therefore, if both loads read the initial values, neither acquire load read from the other thread's release store

Note: The standard explicitly restricts stores to relaxed, release, or sequentially consistent ordering, and loads to relaxed, acquire, or sequentially consistent ordering

- **Compare-exchange success and failure orders:** Compare-exchange often takes two orders:

```
0  value.compare_exchange_weak(
1      expected,
2      desired,
3      std::memory_order_acq_rel, // Success
4      std::memory_order_acquire  // Failure
5  );
```

On success, it reads the old value and writes the new one, so `acq_rel` is possible.

On failure, it only reads the actual value into expected. Because no write occurred, the failure order cannot contain release semantics. Therefore, release and `acq_rel` are invalid failure orders.

A common pattern is

```
0  while (!head.compare_exchange_weak(  
1      expected,  
2      desired,  
3      std::memory_order_release,  
4      std::memory_order_relaxed  
5  )) {  
6      // expected was updated  
7  }
```

The successful operation publishes data. Failure merely retrieves the current value and does not need synchronization.

- **Atomics do not make an entire algorithm atomic:** Suppose two atomic variables must remain consistent with each other:

```
0  std::atomic<int> account_a{100};  
1  std::atomic<int> account_b{100};  
2  
3  void transfer() {  
4      account_a.fetch_sub(50);  
5      account_b.fetch_add(50);  
6  }
```

Each operation is atomic, but the complete transfer is not. Another thread could observe:

```
0  account_a == 50  
1  account_b == 100
```

If both values must change as one transaction, use a mutex:

```
0  #include <mutex>  
1  
2  int account_a = 100;  
3  int account_b = 100;  
4  std::mutex accounts_mutex;  
5  
6  void transfer() {  
7      std::lock_guard lock(accounts_mutex);  
8  
9      account_a -= 50;  
10     account_b += 50;  
11 }
```

Atomics protect individual atomic operations. They do not automatically protect larger invariants involving several operations or objects.

- **Lock-free does not always mean faster:** An atomic type may be implemented directly using hardware instructions, or the implementation may internally use locks.

You can inspect this for a type

```
0  std::atomic<long long> value{0};
1
2  if (value.is_lock_free()) {
3      // This atomic object is implemented without locks.
4  }
```

For a compile-time check when available:

```
0  static_assert(std::atomic<int>::is_always_lock_free);
```

Simple types such as bool, pointers, and properly aligned machine-sized integers are commonly lock-free, but portable code should not assume this unless the implementation guarantees it.

- **Atomic pointers:** Pointers can also be atomic:

```
0  int values[10];
1
2  std::atomic<int*> pointer{values};
3
4  pointer.fetch_add(1); // Advances by one int
5  int* current = pointer.load();
```

Atomic pointers are important in lock-free data structures, although safe memory reclamation makes those structures substantially more complicated than they initially appear.

- **C++20 waiting:** C++20 allows a thread to wait for an atomic value to change without manually spinning continuously:

```
0  std::atomic<bool> ready{false};
1
2  void consumer() {
3      ready.wait(false);
4
5      // ready is no longer false
6  }
7
8  void producer() {
9      ready.store(true);
10     ready.notify_one();
11 }
```

The Embedded Memory Model

- **Intro:** Embedded C++ uses the same language-level memory model as desktop C++, but embedded systems usually adopt a different allocation policy

Allocate everything at startup or compile time, then avoid dynamic allocation during normal operation.

Heap use is discouraged in embedded software because dynamic memory allocation introduces unpredictable behavior, memory fragmentation, and fatal crash risks that violate safety-critical requirement

Functions like `malloc()` or `new` take variable amounts of time to search for free memory blocks, which ruins real-time deadlines where timing must be exact

Allocating and freeing differently sized blocks over time breaks RAM into small, unusable gaps. A system can report plenty of free memory overall yet still fail a new allocation request

Small embedded devices have strict RAM limits. Memory leaks or unexpected allocation spikes cause the heap to run out, leading to hard faults or system resets

The stack and the heap grow toward each other in RAM; when they collide, the system crashes

- **Typical MCU memory layout:** A bare-metal firmware image commonly uses memory like this:

```
Flash
|-- Vector table
|-- Machine code (.text)
|-- Constants (.rodata)
|-- Initial values for global variables

SRAM
|-- Initialized globals/statics (.data)
|-- Zero-initialized globals/statics (.bss)
|-- Heap          usually grows upward
|-- Unused RAM
|-- Stack         usually grows downward
```

Code and constant data normally reside in flash, while mutable data, stacks, and any heap reside in SRAM.

- **Static-storage objects:** Objects can be allocated globally or as static objects. Their storage is reserved before the program begins normal execution. There is no runtime allocation and no fragmentation.

The downside is that these objects exist for the firmware's entire lifetime and may complicate initialization or testing if overused.

- **Automatic stack allocation:** Short-lived objects are normally local variables:

```

0 void process_message() {
1     std::array<std::byte, 64> message{};
2     Parser parser{message};
3
4     parser.process();
5 }

```

The compiler adjusts the stack pointer when the function begins and restores it when the function returns. There is no allocator involved, so allocation is fast and deterministic.

However, MCU stacks are limited. Avoid putting very large buffers on the stack:

Large buffers are generally better placed in static storage or a deliberately managed memory region.

- **Fixed-capacity containers:** Instead of containers that grow dynamically, embedded code often uses containers whose maximum capacity is known at compile time:

```

0 template<typename T, std::size_t Capacity>
1 class StaticVector {
2     std::array<T, Capacity> storage_;
3     std::size_t size_ = 0;
4
5     public:
6     bool push_back(const T& value)
7     {
8         if (size_ == Capacity)
9             return false;
10
11         storage_[size_++] = value;
12         return true;
13     }
14 };

```

This provides vector-like behavior without heap allocation.

- **Fixed-size memory pools:** When objects must be created and destroyed dynamically, a fixed pool is often preferable to a general-purpose heap:

```

0 struct Message {
1     std::uint32_t id;
2     std::array<std::byte, 32> payload;
3 };
4
5 std::array<Message, 16> message_pool;

```

A pool maintains a collection of equally sized slots. Allocation selects a free slot, and deallocation returns it.

- Fixed maximum memory usage
- No external fragmentation
- Usually constant or bounded allocation time
- Straightforward failure behavior
- Easier testing of exhaustion

- **Arena allocation:** An arena owns a fixed region and dispenses memory sequentially:

```
0  class Arena {
1      std::byte* current_;
2      std::byte* end_;
3
4      public:
5      Arena(void* memory, std::size_t size)
6          : current_{static_cast<std::byte*>(memory)},
7            end_{current_ + size} {}
8
9      void* allocate(std::size_t bytes);
10 };
```

Individual objects usually are not freed. The complete arena is reset when all objects allocated from it are no longer needed.

- **New on bare metal:** The compiler still understands

```
0  Widget* object = new Widget{};
```

Conceptually, this performs two operations:

- Obtain raw storage through operator new.
- Construct Widget in that storage.

```
0  void* memory = operator new(sizeof(Widget));
1  Widget* object = ::new (memory) Widget{};
```

Whether ordinary new works depends on the runtime and linker configuration. In many bare-metal toolchains, the allocator ultimately obtains heap space through a low-level function such as `_sbrk`, which you or the vendor runtime must provide.

You can also disable or forbid heap use by:

- Not implementing the required heap-growth routine
- Overriding global operator new
- Treating allocation calls as link errors

- Enforcing a project rule against dynamic containers
- Using linker symbols to reserve zero or limited heap space

`::new` is a C++ expression that forces the compiler to call the global operator `new` function instead of any custom, class-specific operator `new` override

- **Placement new:** Placement new constructs an object in storage you already own:

```
0  #include <new>
1  #include <stddef>
2
3  alignas(Widget) std::byte storage[sizeof(Widget)];
4
5  Widget* widget = new (storage) Widget{};
6
7  widget->~Widget();
```

Placement new does not allocate memory. It only begins the object's lifetime in the supplied storage.

Alignment and object lifetime must be handled correctly, so higher-level wrappers such as `std::optional`, `std::variant`, or a well-tested pool are generally preferable.

Bit manipulations

- Reverse bit pattern of a 32 bit integer (Iterative):

```
0  std::uint32_t rev(std::uint32_t n) {
1      std::uint32_t res{0};
2      for (size_t i{}; i < 32; ++i) {
3          res = res << 1 | (n & 1);
4          n >>= 1;
5      }
6
7      return res;
8  }
```

- Reverse bit pattern of a 32 bit integer (fast):

```
0  unsigned int reverse_bits_fast(unsigned int n) {
1      // Swap adjacent single bits (01010101 and 10101010)
2      n = ((n & 0x55555555) << 1) | ((n & 0xAAAAAAAA) >> 1);
3      // Swap adjacent 2-bit pairs (00110011 and 11001100)
4      n = ((n & 0x33333333) << 2) | ((n & 0xCCCCCCCC) >> 2);
5      // Swap adjacent 4-bit nibbles (00001111 and 11110000)
6      n = ((n & 0x0F0F0F0F) << 4) | ((n & 0xF0F0F0F0) >> 4);
7      // Swap adjacent 8-bit bytes
8      n = ((n & 0x00FF00FF) << 8) | ((n & 0xFF00FF00) >> 8);
9      // Swap the 16-bit halves
10     n = (n << 16) | (n >> 16);
11     return n;
12 }
```